

— ANIPIS · SPRINT 0 REPORT

— — Sprint 0 Backend Hardening.

4 SPIKES implementados · 71 testes novos · 695/695
passing · 0 regressão. Audit cryptographic chain ·
Langfuse observability · Model metadata · Crisis protocol
tests. Ready for Fase 2.

99-SYNTHESIS · 2026-05-16 · @DEV DEX OWNER

Sprint 0 — Backend Hardening Report

Projeto: Anipis · Companion clínico-AI saúde mental BR **Período:** 2026-05-16 (1 sessão) **Owner:** @dev Dex (Constitution gate) **Status:**  COMPLETO · 4/4 SPIKES entregues · 695/695 tests passing

01 Executive Summary

Sprint 0 foi a fase de **endurecimento cirúrgico do backend** Anipis antes de plugar o Concierge MVP (30/Mai-13/Jun). O backend, descoberto em audit como ~70% pronto pra MVP, recebeu 4 layers críticas convergentes com as recomendações de 4 mind clones (Lucia Savage, Atul Butte, Werner Vogels, Demis Hassabis) e 1 spec interna (@dev Dex).

Princípio guia: não refazer o que existe e funciona. Adicionar **somente o que protege contra os 3 maiores riscos** identificados pelo squad-16mai:

- 1. **Tampering em audit trail** (Lucia: LGPD Art. 16 + Werner: Dynamo lesson)
- 2. **Drift de modelo sem detecção** (Atul: RWE pipeline + temporal drift KS-test)
- 3. **Cegueira observacional em produção** (Demis: única adição stack aprovada — Langfuse)
- 4. **Crisis routing sem testes** (Alison: vidas em jogo)

Resultado: **+71 testes / +590 LOC produção / 2 migrations / 1 dependência** ([langfuse@3.38.20](#)). Zero regressão na suite existente. Backend agora preparado pra ANPD audit + temporal drift detection + crisis path validation.

02 SPIKE-1 · Hash Chain Audit Log

Specs convergentes: Lucia Savage (LGPD vs immutability) + Atul Butte (RWE data integrity) + Werner Vogels (Dynamo lesson) + Demis Hassabis (reproducibility AI safety).

O que foi feito

Adicionada camada criptográfica de hash chain `sha256(prev_hash || canonical_repr)` por cima da tabela `audit_events` existente (já APPEND-ONLY desde SAI-SEC-01).

Cada nova linha agora carrega:

- `prev_hash` — referência hex sha256 da linha anterior
- `current_hash` — hex sha256 da linha atual + linkagem
- `hash_algorithm` — algoritmo usado (sha256 default, futuro-proof)

Qualquer tampering em rows intermediárias é detectável em $O(n)$ via a SQL function

```
verify_audit_chain(user_id, from, to) .
```

Por que importa

- **LGPD Art. 16** exige direito ao esquecimento, mas audit log é immutable 5y — hash chain resolve o paradoxo: PII fica em vault separado (crypto-shred), audit log mantém integridade matemática.
- **ANPD audit + Defensoria Pública civil class action** ficam defensáveis com prova matemática de não-tampering.
- **CFM 2.454/2026** exige rastreabilidade clínica completa — hash chain fornece evidência cripto-forte.

Componentes

- Migration `20260516_audit_events_hash_chain.sql` (167 LOC) — 3 colunas + 2 SQL functions + backfill loop + 4 constraints + 2 índices
- `AuditTrailService.flush()` reescrito para computar chain sequencial pré-INSERT
- `AuditTrailService.verifyChain()` novo — invoca SQL function, retorna primeira row corrompida se detectar
- `canonicalRepr()` + `sha256Hex()` static helpers — match exato com SQL function
- `fetchLastHash()` — continuity across processos/restarts

Testes

12 testes novos em `audit-trail-hash-chain.test.ts`:

- Determinismo de `canonical_repr`
- sha256 vector "abc"
- Chain assignment (first row prevHash=null, batch sequencial, continuity)
- 2 entries idênticas geram hashes diferentes (UUID+timestamp diferem)
- `verifyChain` returns intact

03 SPIKE-2 · Langfuse Self-Host Observability

Spec: Demis Hassabis — "Adicionar APENAS Langfuse self-host shadow eval pro MVP. Rejeitar LangGraph, MemO, Letta — backend custom + pgvector existente é suficiente."

O que foi feito

Wrapper singleton `LangfuseClientService` que instrumenta o `LLMRouter` existente com tracing + generations + token usage + latency. Design crítico:

- **Graceful no-op total** quando `LANGFUSE_ENABLED=false` (default). Zero impacto em produção sem decisão explícita de ativar.
- **Dynamic SDK import** — `await import('langfuse')` lazy. Projetos sem a dep não crasham.
- **PII redaction OBRIGATÓRIA pré-Langfuse** — `redactForObservability()` aplica `stripPii()` em CADA input/output antes de enviar. Lucia spec.
- **Apenas últimas 2 messages** enviadas (system + last user) — privacy + payload size.
- **Graceful shutdown** integrado ao SIGTERM/SIGINT do server existente.

Por que importa

- **Demis canonical:** observability é **prerequisite** de eval reproducibility. Sem trace, não tem regression detection.
- **Werner Vogels canonical:** "everything fails all the time" — sem observability, falhas são invisíveis até virar incidente.
- **Atul RWE complementa SPIKE-1+3:** traces ligados a model_card_version + prompt_hash permitem temporal correlation entre prompt change → safety drift.

Componentes

- `services/observability/langfuse-client.ts` (245 LOC NEW) — wrapper + handles + helpers
- `config/env.ts` (+5 env vars) — LANGFUSE_HOST/PUBLIC_KEY/SECRET_KEY/ENABLED + MODEL_CARD_VERSION
- `services/llm/llm-router.ts` (+85 LOC) — `callOpenAI()` e `callAnthropic()` agora opcionalmente recebem `traceOpts` e criam Langfuse generation com PII redacted + token usage + latency
- `server.ts` (+2 LOC) — `langfuseClient.shutdown()` antes de `app.close()`

Testes

14 testes novos em `langfuse-client.test.ts`:

- `isEnabled()` retorna false sem config
- `startTrace()` retorna no-op handle quando disabled
- `generation()`, `span()`, `update()` no-ops são silenciosos
- `flush()` e `shutdown()` são silenciosos quando disabled
- `redactForObservability()` strip CPF / EMAIL / TELEFONE / NOME
- Multi-PII redaction em mensagem única
- Mental health content (palavras como "triste", "ansiosa") preservado

Ativação em produção

Quando decidido (Magalu / RD / Vercel deployment target), set 4 env vars + restart API. Dashboard Langfuse mostra traces de cada chat completion com PII-redacted input/output, token usage, latency, e fallback path.

04 SPIKE-3 · Model Metadata Capture

Spec: Atul Butte — "model_card_version + prompt_hash + embedding_model_id + data_quality_flag em CADA row habilita temporal drift detection KS-test semanal, prompt versioning audit, embedding migration tracking, RWE filtering."

O que foi feito

Adicionadas 4 colunas em `audit_events` (todas NULLABLE para backward-compat):

- `model_card_version` — snapshot do model card ativo (prompts + safety thresholds + LLM choices) no momento do evento. Default vem de env var `MODEL_CARD_VERSION`.
- `prompt_hash` — sha256 do template ID + version usado. Filtra eventos produzidos por prompt versions específicos.
- `embedding_model_id` — qual embedding model usado em events que tocam memory/RAG. NULL pros que não tocam.
- `data_quality_flag` — `clean / partial / degraded / suspect`. RWE pipeline filtra antes de export.

Chain recompute estratégia

Como `canonicalRepr` mudou (v1 → v2 com 4 novos fields appended), a migration **recomputa o hash chain inteiro** em ordem cronológica usando v2. Isso invalida hashes pré-SPIKE-3 mas mantém chain integrity. `verify_audit_chain()` pós-migration passa.

Auto-derivation de `dataQualityFlag`

`recordLlmDecision()` agora deriva automaticamente:

- `wasFiltered=true` → `dataQualityFlag='partial'`
- caso normal → `'clean'`
- caller pode override com qualquer um dos 4 enum values

Helper `hashPrompt`

`AuditTrailService.hashPrompt(templateId, version)` estático — gera sha256 determinístico de `${templateId}@${version}`. Use em `chat-service.ts` quando logar LLM decisions pra que RWE pipeline rastreie qual prompt version produziu cada evento.

Por que importa

- **CONEMO-style RCT** (Atul + @analyst Atlas D-03 recommendation) exige rastreio rigoroso de model versions por temporal slice — sem isso, peer-review rejeita publicação.
- **Temporal drift detection** (KS-test semanal sobre safety event rate por model_card_version) só funciona se cada row carrega o version snapshot.
- **ANS RN-627 audit** futuro (Q4/2026+) vai exigir prova de qual modelo decidiu qual encaminhamento — RWE pipeline blindado.

Testes

18 testes novos em `audit-trail-model-metadata.test.ts`:

- 4 fields default NULL
- `record()` aceita todos via options
- 4 `DataQualityFlag` values aceitos
- Auto-derivation flag no `recordLlmDecision` (clean vs partial)
- `hashPrompt` determinístico e sensível a `templateId` + `version`
- `canonicalRepr` v2 contém metadata fields
- Hash chain ainda linkado quando metadata varia

05 SPIKE-4 · Crisis Protocol Tests + Coverage Baseline

Specs convergentes: Dex (cobertura crítica), Quinn (gate 22 Tier S), Alison Darcy (Mr. Walker protocol — vidas em jogo).

Coverage baseline pós-Sprint 0

Statements: 24.95% (3087 / 12370)
Branches: 86.83% (620 / 714)
Functions: 92.30% (228 / 247)

Interpretação: branches + functions altos indicam que arquivos com testes estão excelentes. Statements baixo indica que muitos arquivos ainda não têm testes (rotas E2E, services não-críticos).

Coverage por path crítico

PATH	COVERAGE	STATUS
safety/audit-trail.ts	79.67%	GOOD (LGPD audit)
safety/content-classifier.ts	100%	PERFECT (safety)
safety/injection-guard.ts	94.70%	GOOD (prompt injection)
safety/response-validator.ts	100%	PERFECT (output sanity)
safety/risk-scorer.ts	94.88%	GOOD (risk math)
llm/output-filter.ts	89.87%	GOOD
llm/safety-classifier.ts	93.36%	GOOD
pii-stripper.ts	100%	PERFECT (LGPD Art. 11)
companion/empathy-engine.ts	99.29%	PERFECT
crisis-protocol-service.ts	+27 tests	CRITICAL paths covered
llm-router.ts	0% → backlog Sprint 1	GAP
chat-service.ts	0% → backlog Sprint 1	GAP

Crisis Protocol Service — 27 testes novos

Foco em paths críticos onde vida está em jogo:

- **classificationToColor** — todos 5 níveis (none/low/medium/high/critical) + fallback unknown → green (SAFE default)
- **Green protocol** — não bypass LLM, não notifica frontend
- **Yellow protocol** — prompt injection, não bypass, notifica
- **Orange protocol** — hybrid (DB response + prompt injection), notifica, com fallback in-memory quando DB vazio + fallback degraded quando DB throws
- **RED protocol (CRÍTICO)** — sempre bypassa LLM, retorna pre-validated content, fallback chain triplo (DB → CRISIS_RESPONSES const → HARDCODED_CVV_FALLBACK)
- **personalizeResponse XSS-safe** — sanitiza `<script>`, `{`, `}`. Trunca para 50 chars. Empty userName preserva placeholder.
- **buildCrisisWSMessage** — null para green, formatado para red
- **logCrisisEvent fire-and-forget** — nunca throws mesmo com DB down
- **alertEmergencyContact** — false quando sem primary, true quando encontrado, PII mascarada em logs

Documentação adicional

`apps/api/COVERAGE-PLAN.md` (novo) — baseline + targets B0 (Sprint 0 — atingido) / B1 (pre-Beta 2026-07-31) / B2 (post-Beta + Series A ready 2026-12-31). Lista 10 gaps documentados pra Sprint 1+.

06 Stats Consolidados

Diff código

CATEGORIA	QUANTIDADE
Migrations SQL novas	2
Colunas adicionadas em <code>audit_events</code>	7
SQL functions novas	2 (canonical_audit_repr v2, verify_audit_chain)
Source files modificados	4
Source files novos	1 (langfuse-client.ts)
Test files novos	4
Test cases novos	71 (12 + 14 + 18 + 27)
Total tests passing pós-Sprint	695 / 695
Test files total	25
LOC produção adicionado	~590
LOC tests adicionado	~980
TS dependencies novas	1 (langfuse@3.38.20)

Tempo

- Sprint 0 duração: 1 sessão (mesma data de reinício do projeto)
- Concierge MVP gate D+7: 13/Jun/2026 (4 semanas)
- CFM 2.454/2026 enforcement: ago/2026 (~12 semanas)
- Buffer pós-launch: 4-6 semanas

Custo

- Cash imediato: **R\$0** (sem deploy de Langfuse self-host ainda)
- Cash projetado Sprint 1 quando deploy: R\$50-200/mês infra Langfuse self-host
- Dependency upgrade risk: baixo (Langfuse v3.38 estável, OpenAI/Anthropic SDKs unchanged)

07 O Que Isso Destrava

Para o Concierge MVP (30/Mai-13/Jun)

Backend tem agora **safety net mínima viável** para o piloto humano:

- Toda decisão clínica do sistema fica auditada com hash chain (Lucia DPIA defensável)
- Mr. Walker protocol RED bypass tem 27 testes cobrindo (Alison spec atendida)
- Crisis routing fallback triplo (DB → in-memory → hardcoded CVV) testado
- Observability hook pronto pra plugar quando Langfuse self-host deployado

Para Sprint 1 (build core, semanas 3-6 pós-Concierge)

3 spikes documentados como next priority em `COVERAGE-PLAN.md`:

1. `llm-router.ts` integration tests (mock OpenAI + Anthropic + Langfuse assertions)
2. `chat-service.ts` E2E happy + crisis + injection paths
3. `clinical-deterioration-service.ts` 3/7/30-day window logic

Para auditoria externa

- **ANPD audit** (12 meses pós-launch, alta probabilidade): hash chain + PII vault separado + retention 5y inviolable = defensável
- **CFM 2.454/2026 review** (ago/2026 enforcement): model_card_version + prompt_hash por evento clínico = rastreabilidade total
- **RCT peer-review** (modelo CONEMO Indaiatuba, Atul spec): RWE pipeline blindado com data_quality_flag + temporal drift detection harness pronto

08 Pré-requisitos Operacionais (não-código)

Para Sprint 1 começar smoothly, três decisões humanas pendentes (não bloqueiam Concierge mas bloqueiam Sprint 1):

1. **Aplicar 2 migrations** — `supabase db push` no ambiente correto (staging primeiro, prod após smoke test)
2. **Decidir Langfuse self-host deployment target** — Magalu / RD Cloud BR / Railway / Vercel? (latência BR + custo + LGPD residency tradeoff)

3. Definir **MODEL_CARD_VERSION** rotation policy — manual snapshot quando prompts mudam OU auto-derived de git tag? (afeta como temporal drift correlation funciona)

Esses 3 ficam no backlog @devops para Sprint 1 kick-off.

09 Verdict Sprint 0 → Sprint 1

Sprint 0 status:  **READY FOR FASE 2**

Backend Anipis está production-blocked apenas por decisões operacionais humanas, não código.

Nenhuma das 4 SPIKES adicionadas tem testes vermelhos. Suite completa (695 tests / 25 files) passa em 4.92s.

Fase 2 candidatos (próxima escolha do founder)

Depois do Sprint 0 fechado, há 4 frentes igualmente válidas pra Fase 2:

- **A.** Frontend Rebrand v2 implementação (Daria + Pixel + Flow specs prontas em REBRAND-PACK-v1)
- **B.** Schema migrations Sprint 0-3 completas (12 migrations Dara spec — pii_vault + crisis_events superset + RWE event schema)
- **C.** Eval harness + golden set 500 cenários PT-BR (Atul: 200 piso / 500 estratificado / 1250 ideal)
- **D.** Sprint 1 backlog destravado (3 integration test spikes do COVERAGE-PLAN.md)

Recomendação Orion: **A (rebrand v2) + C (eval harness)** em paralelo, porque ambos têm specs completas e zero overlap com Concierge MVP (que roda 30/Mai-13/Jun).

Orion · Master Orchestrator AIOS · 2026-05-16 Sprint 0 backend hardening · Anipis Companion Clínico-AI Saúde Mental BR